

Portfolio

Name. 장하영

E-mail. hayoungtuk@gmail.com



Chapter 1

Introduction

(사진)

Profile

Name 장하영
Tel 010 2139 9893
E-mail hayoungtuk@gmail.com

Education

2016 오륜중학교 졸업
2019 창덕여자고등학교 졸업
2026 한국공학대학교 졸업예정

Skills

C/C++	IOCP	Windows API	Git
JAVA	Multithread		
Rust	Ms SQL		
Python			

Contents

Project 1

졸업 작품



Project 1

졸업 작품

장르 : 3D TPS 5:5 PvP 히어로 슈팅 게임

개발기간 : 2024/07 - 2025/07

개발 언어 : Rust

개발 인원 : 3명 (클라이언트 1명, 서버 2명)

개발 환경: Windows, macOS, tokio, wgpu,
Visual Studio Code, git

중점 연구 분야: Rust/Tokio 비동기 프로그래밍 활용

담당 역할: 서버 개발

- UDP/TCP 혼용 구현
- 리스폰/거점 점령/캐릭터별 자원 관리
- NPC 플레이어 구현

Git : https://github.com/HK416/capstone_project

Youtube : <https://youtu.be/64DGuo1Z5No?si=rdXAVJfscwa7vh28>

Rust 선택 사유

Rust 는 자원의 소유권을 관리하는 방식으로 메모리와 공유자원에서 발생할 수 있는 문제를 컴파일 타임에 체크할 수 있는 장점을 가지고 있다. 또한, 타입 시스템을 통해 스레드 간의 자원의 이동과 공유를 제한할 수 있기 때문에, C/C++ 와 비슷한 수준의 성능을 유지하면서도 더 안전한 것을 앞세워 등장하기도 했다. 이러한 특성에서 멀티스레드나 높은 성능을 필요로 하는 게임 개발에 사용해볼 수 있지 않을까 싶어 프로젝트에서 활용해 보기로 하였다.



Project 1

졸업 작품

UDP/TCP 혼용 구현

서버 시작 시 TCP와 동일 주소로 UDP 바인딩.
수신 루프(udp_packet_receive_loop): recv_from → RawPacket 파싱 → SessionManager로 전달.
송신 루프(udp_packet_send_loop): Queue에서 (addr, RawPacket) 팝 → send_to.
세션 생성(wait_for_players): TCP accept 시 Session::new(addr, udp_sender)로 UDP 송신 큐 주입.

Tokio 라이브러리를 이용한 비동기 I/O: Session thread의 작업과 무관하게 read/write 할 수 있도록 별도 task(user-level thread)로 분리하여 처리.
Lock-Free Queue로 스레드 사이 데이터/메시지 전달: World thread에서 Session thread로 상태 업데이트 메시지 전달/Session thread에서 World thread로 이벤트

UDP 수신 루프

recv_from 시 RawPacket → 세션 큐로 전달

```
async fn udp_packet_receive_loop(socket: Arc<UdpSocket>) {
    let mut buf = [0; 1024];
    loop {
        buf.fill(0);
        match socket.recv_from(&mut buf).await {
            Ok((size, addr)) => {
                let received_data = &buf[..size];
                match RawPacket::try_from_bytes(received_data) {
                    Ok(packet) => {
                        if let Some(session) = SessionManager::get(&addr) {
                            session.add_received_packet(packet);
                        }
                    }
                    Err(e) => {...}
                }
            }
            Err(e) => {...}
        }
        tokio::time::sleep(tokio::time::Duration::from_micros(1)).await;
    }
}
```

run_server

TCP/UDP 바인드 및 루프 기동

```
let udp_socket = match UdpSocket::bind(addr).await {
    Ok(socket) => Arc::new(socket),
    Err(e) => {...}
};
let udp_sender = Arc::new(Queue::new());
let udp_recv_socket = udp_socket.clone();
tokio::spawn(udp_packet_receive_loop(udp_recv_socket));

let udp_send_socket = udp_socket.clone();
let udp_sender_clone = udp_sender.clone();
tokio::spawn(udp_packet_send_loop(udp_send_socket, udp_sender_clone));

wait_for_players(listener, udp_sender).await;
```

UDP 송신 루프

```
async fn udp_packet_send_loop(
    socket: Arc<UdpSocket>,
    udp_sender: Arc<Queue<(SocketAddr, RawPacket)>>,) {
    loop {
        if let Some((addr, packet)) = udp_sender.pop() {
            let packet_data = packet.as_bytes();
            if let Err(e) = {...}
            tokio::time::sleep(tokio::time::Duration::from_micros(1)).await;
        }
    }
}
```

Project 1

졸업 작품

리스폰/거점 점령/캐릭터별 자원 관리 - 리스폰

```

if h.health_data.shield < final_damage {
  final_damage -= h.health_data.shield;
  h.health_data.shield = 0;
  if h.health_data.remaining <= final_damage {
    // 플레이어 행동 불능 처리
    h.health_data.remaining = 0;
    h.action_state = ActionState::Retreat;
    h.action_state_timer.0 = 0;
    h.movement_state = MovementState::Idle;
    h.movement_state_timer.0 = 0;
    h.action_notify = ActionNotify::Retreat;

    // 플레이 데이터 갱신
    h.retreat_count += 1;
    s.kill_count += 1;
  } else {
    h.health_data.remaining -= final_damage;
  }
}

```

사망 처리

```

ActionEvent::Respawn => {
  // 해당 플레이어 데이터를 가져옵니다.
  let data = match world.players.get_mut(&uid) {
    Some(data) => data,
    None => {...}
  };

  // 플레이어 위치와 상태를 초기화합니다.
  let team = data.team();
  let team_index = data.team_index();
  let (rotation, translation) = match team {
    Team::Blue => (
      stage_attributes.blue_team_rotation,
      stage_attributes.blue_team_positions[team_index],
    ),
    Team::Red => (
      stage_attributes.red_team_rotation,
      stage_attributes.red_team_positions[team_index],
    ),
  };
  let direction = rotation.mul_vec3a(glam::Vec3A::Z);
  let longitude = glam::Quat::IDENTITY.angle_between(rotation);

  data.health_data.shield = 0;
  data.health_data.remaining = data.health_data.num_maximum_health();
  data.bullet_data.remaining = data.bullet_data.num_maximum_bullets();
  data.skill_cost_data.remaining = 0;
  data.skill_cost_timer = 0;
  data.input_state_timer.0 = 0;
  data.rotation = rotation;
  data.velocity.0 = glam::Vec3A::ZERO;
  data.direction.0 = direction;
  data.translation = translation;
  data.set_grounded(true);
  data.set_invincible(true);
  data.latlon = LatLon::new(10f32.to_radians(), longitude);
}

```

Respawn 블록

이벤트 처리: ActionEvent::Respawn 블록
 사망 처리: hit_player() 내 사망 시 상태 전환

흐름

피격 처리(hit_player)로 체력 0 이하
 → ActionState::Retreat, 이동/행동 타이머 리셋,
 킬/퇴각 카운트 갱신

이후 ActionEvent::Respawn 발생
 팀/슬롯으로 스폰 위치·회전 결정
 자원/상태 초기화

패킷 연동

리스폰 후 상태는 Pull/Status 패킷에 반영됨
 - Pull: InGamePullPacket에 위치/회전/행동 상태
 - Status: InGameStatusPacket에 체력/총알/스킬 수치

Project 1

졸업 작품

리스폰/거점 점령/캐릭터별 자원 관리 - 거점 점령

```
pub fn update<'a, I>(&mut self, players: I, elapsed_time_ms: u16)
where
    I: Iterator<Item = &'a Player>,
{
    // 점령지 안에 있는 플레이어 수를 계산합니다.
    let mut num_blue_players = 0;
    let mut num_red_players = 0;
    for player in players {
        if player.action_state != ActionState::Retreat {
            if self.collider.check_point_collision(&player.translation) {
                match player.team() {
                    Team::Blue => {
                        num_blue_players += 1;
                    }
                    Team::Red => {
                        num_red_players += 1;
                    }
                }
            }
        }
    }

    if num_red_players > 0 && num_blue_players == 0 {
        let offset = num_red_players as u16;
        self.capture_point.set_capture_team(Some(Team::Red));
        self.capture_point.update(elapsed_time_ms, offset);
    }
    else if num_blue_players > 0 && num_red_players == 0 {
        let offset = num_blue_players as u16;
        self.capture_point.set_capture_team(Some(Team::Blue));
        self.capture_point.update(elapsed_time_ms, offset);
    }
    else {
        self.capture_point.set_capture_team(None);
    }
}
```

점령 진행
갱신

```
if self.play_elapsed_time_ms >= MAX_GAME_TIME {
    // 현재 점령 점수가 높은 팀을 가져옵니다.
    let capture_point = self.capture_point.as_ref();
    let max_score_team = capture_point.max_score_team();
} else {
    // 현재 점령 점수가 높은 팀을 가져옵니다.
    let capture_point = self.capture_point.as_ref();
    if capture_point.red_score() >= 1.0 {...}
    if capture_point.blue_score() >= 1.0 {...}
}
```

승리 판정

```
let mut packet =
    InGameStatusPacket::new(self.capture_point.as_ref().clone(), players, vec![], vec![]);

// 패킷을 전송합니다.
let mut damage_logs: Vec<_> = self.damage_logs.drain(..).collect();
let mut removed_bullets: Vec<_> = self.removed_bullets.drain(..).collect();
loop {
    let count = damage_logs.len().min(MAX_IN_GAME_LOGS);
    packet.damage_logs = damage_logs.drain(..count).collect();

    let count = removed_bullets.len().min(MAX_IN_GAME_BULLETS);
    packet.removed_bullets = removed_bullets.drain(..count).collect();

    for session in world.sessions.keys() {
        session.tcp_write(packet.as_raw());
    }
    if damage_logs.is_empty() && removed_bullets.is_empty() {
        break;
    }
}
```

점령/스코어 전달 패킷

흐름

스테이지 로드 → CapturePointObject 생성
 → 매 틱에 구역 내 인원 집계 및 진행도 갱신
 → Status 패킷으로 클라이언트 HUD 동기화
 → 시간 초과 또는 스코어 충족 시 승리 전환.



Project 1

졸업 작품

리스폰/거점 점령/캐릭터별 자원 관리 - 캐릭터별 자원 관리

```
pub struct BulletData {
    pub fires_per_attack: u16,
    pub remaining: u16,
    maximum: u16,
}

impl BulletData {
    pub const fn new(remaining: u16, maximum: u16) -> Self { ... }
    pub fn num_maximum_bullets(&self) -> u16 { self.maximum }
    //직렬화/역직렬화 등...
}
```

탄약 데이터 구조(간략)

```
update_action_state_timer(
    uid,
    player.held_input,
    &mut bullet_data,
    &mut skill_cost_data,
    &mut action_state,
    &mut action_state_timer,
    character_attributes,
    elapsed_time_ms,
    &mut events,
);
```

공격 처리 시 탄약 사용

```
players.push(InGamePlayerStatusPullData::new(
    uid,
    data.kill_count,
    data.retreat_count,
    data.health_data.shield,
    data.health_data.remaining,
    data.bullet_data.remaining,
    data.skill_cost_data.remaining,
    data.permission(),
    connected,
    data.is_invincible(),
    data.network_state(),
));
```

상태 전송(잔탄 포함)

입력 수신 → 상태 전이

- handle_input_event → update_action_state 호출로 행동/이벤트 생성
- 스킬 진입 시 즉시 코스트 차감

이벤트 스케줄/처리

- 서버 틱(update)에서 update_action_state_timer로 이벤트 스케줄링

이벤트 처리

- Attack → 총알 생성
- Reload → 탄약 최대치로 보충
- Skill → 스킬 사용 로직 실행

틱별 자원 갱신은 update_player에서

- 스킬 코스트 자연 회복(SKILL_COST_TICK)
- update_action_state_timer로 실제 탄약/코스트 소모 반영
- 일반 탄환 적중 시 슈터의 스킬 코스트 + 10 (상한까지)

리스폰 시

- 탄약/스킬 코스트/체력 초기화

상태 전송

- InGameStatusPacket으로 현재 탄약/스킬 코스트 브로드캐스트

Project 1

졸업 작품

NPC 플레이어 구현(봇전)

A* 경로탐색 + 그리드맵 기반 보행성·장애물 회피 + 방문기록/탈출 휴리스틱 + 자동 공격으로 동작함

주요 알고리즘 코드

```
pub fn grid_based_astar_pathfind( start: Vec3A, goal: Vec3A,
    grid_map: &GridMap2D,) -> Option<Vec<Vec3A>> {

    let distance = (goal - start).length();
    if distance > 50.0 {
        let is_walkable = |pos: Vec3A| grid_map.is_walkable(pos);
        hierarchical_pathfind(start, goal, 8.0, 2.0, is_walkable)
    } else {
        grid_based_astar_pathfind_standard(start, goal, grid_map)
    }
}
```

진입 함수
ai_player.rs에서 직접 호출되는 A* 진입점. 장거리면 HPA*
단거리면 표준 A*로 위임.

```
impl GridPos {
    pub fn center_bias_heuristic(&self, other: &GridPos, k:f32)->u32{
        let base = std::cmp::max((self.x-other.x).abs(),
            (self.z - other.z).abs()) *10;
        let center_distance = ((self.x * self.x+self.z*self.z) as f32).sqrt();
        (base as f32 + center_distance * k) as u32
    }
    pub fn adaptive_center_heuristic(&self, other:&GridPos) -> u32 {
        let dx = (self.x - other.x).abs() as u32;
        let dz = (self.z - other.z).abs() as u32;
        let d = std::cmp::max(dx, dz);
        let k = if d < 20 {15.0} else if d>10 {8.0} else {3.0};
        self.center_bias_heuristic(other, k)
    }
}
```

Grid 기반 휴리스틱 알고리즘

```
pub fn grid_based_astar_pathfind_standard(start: Vec3A, goal: Vec3A,
    grid_map: &GridMap2D)-> Option<Vec<Vec3A>> {
    let start_grid = GridPos::from_world_pos(start, grid_map.bounds(),
        grid_map.grid_size());
    let goal_grid = GridPos::from_world_pos(goal, grid_map.bounds(),
        grid_map.grid_size());
    let half_w = (grid_map.dimentions.0/2) as i32;
    let half_h = (grid_map.dimentions.1/2) as i32;

    let result = astar(&start_grid,
        |pos|{
            pos.extended_neighbors()
                .into_iter()
                .filter_map(|(n,base_cost)|{
                    if n.x < -half_w || n.x >= half_w || n.z < -half_h || n.z >= half_h {
                        return None;
                    }
                    let wp = n.to_world_pos_with_bounds(grid_map.grid_size(),
                        grid_map.base_height(), grid_map.bounds());
                    if !grid_map.is_safe_walkable(wp) { return None; }

                    let center = n.center_bias_heuristic(&GridPos::new(0.0), 2.0);
                    let cost= ((base_cost as f32*0.9)+(center as f32*0.1)).max(1.0) as u32;
                    Some((n, cost))
                })
                .collect()
        },
        |pos| pos.adaptive_center_heuristic(&goal_grid),
        |pos| *pos == goal_grid,
    );
    result.map(|(path,_)| path.into_iter()
        .map(|gp| gp.to_world_pos_with_bounds(grid_map.grid_size, grid_map.base_height))
        .collect())
}
```

표준 A*
확장 이웃 + 안전 그리드 필터 + 적응형 중앙 휴리스틱

```
pub fn hierarchical_pathfind<F>( start: Vec3A, goal: Vec3A,coarse_grid: f32,
    fine_grid: f32,
    is_walkable: F,
) -> Option<Vec<Vec3A>>
where
    F: FnMut(Vec3A) -> bool + Clone,{
        coarse_grid,
        fine_grid
    };
let coarse_path = advanced_astar_pathfind(start, goal, coarse_grid,
    is_walkable.clone());

if coarse_path.len() <= 2 { return Some(coarse_path);}
let mut refined_path = Vec::new();
for i in 0..coarse_path.len() - 1 {
    let segment_start = coarse_path[i];
    let segment_end = coarse_path[i + 1];

    if let Some(segment_path) =
        advanced_astar_pathfind(segment_start, segment_end, fine_grid,
            is_walkable.clone())
    {
        if i == 0 { refined_path.extend(segment_path); }
        else {
            refined_path.extend(segment_path.into_iter().skip(1));
        }
    } else {
        if i > 0 { refined_path.push(segment_end); }
    }
}
if refined_path.is_empty() { Some(coarse_path) }
else { Some(refined_path) }
```

장거리용 HPA* 알고리즘

Name. 장하영

E-mail. hayoungtuk@gmail.com



Thank you!